# 영지식증명 톺아보기 1편: 비밀을 증명한다는 것

**Author**: 김대익
**Reviewed By**: —

---

**Disclaimer**: 본 글은 영지식증명(Zero-Knowledge Proof)의 핵심 개념을 개발자 관점에서 풀어낸 시리즈의 첫 번째 편입니다. 수학적 엄밀함보다는 직관적 이해를 우선했습니다.

---

## 1. 들어가며

"내가 비밀번호를 알고 있다는 걸 증명할게. 근데 비밀번호는 안 알려줄 거야."

처음 들으면 말장난 같습니다. 뭔가를 안다는 걸 증명하려면, 그걸 보여줘야 하는 거 아닌가요?

**영지식증명(Zero-Knowledge Proof, ZKP)**은 정확히 이 "불가능해 보이는 일"을 가능하게 합니다. 1985년 Goldwasser, Micali, Rackoff가 처음 제안한 이 개념은, 40년이 지난 지금 블록체인의 핵심 기술이 되었습니다.

이 시리즈에서는 ZKP가 어떻게 작동하는지, 왜 블록체인에서 중요한지, 그리고 Groth16부터 최신 zkVM까지 어떻게 발전해왔는지를 다룹니다. 수식이 나올 때마다 "왜 이 도구를 쓰는가"를 명확히 하겠습니다.

첫 편에서는 마인크래프트 레드스톤 회로를 비유로 ZKP의 핵심 개념을 잡아보겠습니다.

---

## 2. 마인크래프트로 이해하는 영지식증명

### 2.1 거대한 레드스톤 회로

마인크래프트를 해보셨다면 레드스톤 회로를 아실 겁니다. 레버를 당기면 신호가 전달되고, AND 게이트, OR 게이트 같은 논리 회로를 만들 수 있죠.

이런 상황을 상상해봅시다:

> 누군가가 **엄청나게 복잡한 레드스톤 회로**를 만들었습니다. 레버가 100개 있고, 회로를 거쳐서 최종적으로 **다이아몬드 블록 램프**가 켜지거나 꺼집니다.
>
> 이 회로는 특별합니다. 100개의 레버 중 **딱 하나의 조합**만이 다이아몬드 램프를 켭니다. 나머지 2^100 - 1개의 조합은 전부 램프를 끄죠.

자, 이제 질문입니다:

**"나는 다이아몬드 램프를 켜는 레버 조합을 알고 있어. 근데 그 조합은 비밀이야. 어떻게 증명하지?"**

가장 단순한 방법은 직접 레버를 조작해서 램프를 켜 보이는 것입니다. 하지만 그러면 다른 사람들이 레버 상태를 보고 조합을 알아버립니다.

### 2.2 영지식증명이 하는 일

영지식증명은 이 문제를 해결합니다:

1. **나는 레버 조합을 알고 있다** (증명하고 싶은 것)
2. **하지만 레버 조합 자체는 공개하지 않는다** (비밀 유지)
3. **검증자는 내가 진짜 아는지 확신할 수 있다** (신뢰할 수 있는 검증)

여기서 핵심 용어들이 등장합니다:

| 레드스톤 비유 | ZKP 용어 | 설명 |
|---|---|---|
| 레버들의 상태 | **Witness (위트니스)** | 증명자만 아는 비밀 입력값 |
| 레드스톤 회로 | **Circuit (회로)** | 연산을 표현한 제약 조건들 |
| "다이아몬드 램프가 켜졌다" | **Public Output** | 누구나 확인할 수 있는 결과 |
| 회로 설계도 | **Public Input** | 누구나 아는 공개 정보 |

### 2.3 비유를 넘어서: 실제 ZKP의 구조

레드스톤 비유가 좋은 이유는, 실제 ZKP의 **산술 회로(Arithmetic Circuit)**와 구조적으로 비슷하기 때문입니다.

레드스톤에서:

- 레버: ON(1) 또는 OFF(0)
- 게이트: AND, OR, NOT 연산
- 연결: 신호가 흐르는 경로

산술 회로에서:

- 입력: 유한체(Finite Field)의 원소들
- 게이트: 덧셈(+)과 곱셈(×) 연산
- 연결: 값이 전달되는 와이어

차이점은 산술 회로가 0과 1만 다루는 게 아니라 **큰 숫자들의 연산**을 다룬다는 것입니다. 그리고 이게 핵심입니다—모든 컴퓨터 연산은 결국 산술 연산으로 바꿀 수 있거든요.

---

## 3. ZKP의 세 가지 조건

1985년 논문에서 정의한 영지식증명의 세 가지 조건을 레드스톤 비유로 설명하겠습니다.

### 3.1 완전성 (Completeness)

> "진짜 레버 조합을 아는 사람은 **항상** 검증을 통과할 수 있다."

정직한 증명자가 올바른 위트니스를 가지고 있다면, 검증자는 반드시 "참"이라고 판단합니다. 진짜를 진짜라고 인정하는 것.

### 3.2 건전성 (Soundness)

> "레버 조합을 모르는 사람이 검증을 통과할 확률은 **무시할 수 있을 만큼 작다**."

거짓 증명자가 검증자를 속일 확률이 극히 낮습니다. 보통 2^-128 이하, 즉 우주의 원자 수보다 작은 확률입니다.

### 3.3 영지식성 (Zero-Knowledge)

> "검증 과정에서 검증자는 **레버 조합에 대해 아무것도 배우지 못한다**."

검증자가 얻는 정보는 오직 "이 사람이 조합을 알고 있다/모른다" 뿐입니다. 조합의 힌트조차 얻지 못합니다.

이 세 번째 조건이 "영지식"이라는 이름의 유래입니다. 검증자의 지식 증가량이 0(zero)이라는 뜻이죠.

---

## 4. 왜 블록체인에서 중요한가

### 4.1 프라이버시: 금액을 숨기면서 거래 증명

비트코인에서 거래를 하면 모든 정보가 공개됩니다. 누가 누구에게 얼마를 보냈는지 누구나 볼 수 있죠.

ZKP를 사용하면:

- "이 거래는 유효하다" (잔액이 충분하고, 서명이 올바르다)
- 하지만 "누가, 누구에게, 얼마를" 은 숨긴다

Zcash가 2016년에 처음 구현한 방식입니다. 이후 시리즈에서 자세히 다룹니다.

### 4.2 확장성: 1000개 거래를 하나의 증명으로

이더리움 메인넷은 초당 약 15개의 거래를 처리합니다. 모든 노드가 모든 거래를 직접 실행해야 하기 때문이죠.

ZKP를 사용하면:

- L2에서 1000개의 거래를 실행
- "이 1000개 거래가 모두 유효하다"는 증명 하나를 생성
- L1에는 증명만 올림 (수십~수백 바이트)
- L1 노드는 1000개를 다시 실행하는 게 아니라 **증명만 검증**

검증은 원래 연산보다 훨씬 빠릅니다. 이게 **zkRollup**의 핵심 아이디어입니다.

### 4.3 레드스톤 비유로 다시 보기

| 사용 사례 | 레드스톤 비유 |
|---|---|
| Zcash (프라이버시) | "램프가 켜졌다"만 보여주고, 어떤 레버를 당겼는지는 숨김 |
| zkRollup (확장성) | 1000개의 회로 실행 결과를 하나의 "램프 상태"로 압축 |
| zkEVM | 레드스톤 회로가 EVM 전체를 시뮬레이션 |

---

## 5. 산술 회로: 모든 것의 시작

### 5.1 왜 "산술" 회로인가

레드스톤은 AND, OR, NOT 같은 **논리 연산**을 씁니다. 하지만 ZKP에서는 **덧셈과 곱셈**만 씁니다.

왜 그럴까요? 암호학에서 사용하는 수학적 구조(유한체, 타원곡선)가 덧셈과 곱셈 위에서 정의되기 때문입니다. 논리 연산도 산술 연산으로 바꿀 수 있습니다:

- AND: `a × b` (둘 다 1일 때만 1)
- OR: `a + b - a × b`
- NOT: `1 - a`

결국 모든 계산은 덧셈과 곱셈의 조합으로 표현됩니다.

### 5.2 `a × b = c` 형태로 바꾸기

ZKP 시스템들은 보통 연산을 다음 형태로 제한합니다:

```
(왼쪽 입력들의 선형 조합) × (오른쪽 입력들의 선형 조합) = (출력의 선형 조합)
```

더 간단히: **`a × b = c`**

이게 **R1CS (Rank-1 Constraint System)**의 기본 형태입니다. "왜 이렇게 제한하나요?"라고 물을 수 있습니다.

**이유**: 나중에 다항식으로 변환할 때, 이 형태가 가장 다루기 쉽습니다. 곱셈이 딱 한 번만 있으면 2차 다항식으로 깔끔하게 떨어지거든요.

### 5.3 간단한 예제

함수 `f(x) = x³ + x + 5`를 생각해봅시다.

"나는 `f(x) = 35`가 되는 x를 알고 있다"를 증명하고 싶습니다. (답은 x = 3)

이 계산을 `a × b = c` 형태들로 쪼개면:

```
(1) x × x = sym_1       // sym_1 = x² = 9
(2) sym_1 × x = y       // y = x³ = 27
(3) (x + y) × 1 = sym_2 // sym_2 = x + x³ = 30
(4) (sym_2 + 5) × 1 = out // out = 35
```

4개의 제약 조건(constraint)이 생겼습니다.

이제 **문제가 바뀌었습니다**:

> "x = 3일 때 f(x) = 35임을 증명해라"
>
> ↓
>
> "위 4개의 제약 조건을 모두 만족하는 값들을 알고 있음을 증명해라"

비밀(witness)은 `x = 3`이고, 중간 값들 `sym_1 = 9`, `y = 27`, `sym_2 = 30`도 함께 witness에 포함됩니다.

---

## 6. 왜 이렇게 복잡하게 하는가

여기서 자연스러운 질문이 나옵니다:

> "그냥 x = 3 대입해서 35 나오는 거 보여주면 되는 거 아니야?"

맞습니다, 이 예제에서는요. 하지만 실제 ZKP가 풀려는 문제는 다릅니다:

### 6.1 프라이버시가 필요한 경우

Zcash에서 "내 잔액이 100 이상이다"를 증명할 때, 잔액 자체(예: 150)는 숨기고 싶습니다.

### 6.2 검증 비용을 줄이고 싶은 경우

zkRollup에서 1000개의 거래를 검증할 때:

- 직접 실행: 1000개 거래 모두 실행 (느림)
- ZKP 검증: 증명 하나만 확인 (빠름)

증명 생성은 오래 걸리지만, **검증은 원래 계산보다 훨씬 빠릅니다**. 이게 핵심입니다.

### 6.3 앞으로의 여정

지금까지 한 것:

1. 계산을 `a × b = c` 형태의 제약 조건들로 바꿈

앞으로 할 것: 2. 제약 조건들을 **다항식**으로 압축 (3편) 3. 다항식을 **하나의 점**으로 압축 (4편, 커밋먼트) 4. 그 점이 올바른지 **빠르게 검증** (4편, 페어링)

매 단계마다 "왜 이 도구를 쓰는가"를 명확히 할 것입니다.

---

## 7. Interactive vs Non-Interactive

### 7.1 원래 ZKP는 대화형이었다

1985년 논문의 ZKP는 **Interactive Proof**였습니다:

1. 증명자가 메시지를 보냄
2. 검증자가 랜덤 챌린지를 보냄
3. 증명자가 응답
4. 2-3을 여러 번 반복
5. 검증자가 판단

문제: 블록체인에서는 "검증자"가 스마트 컨트랙트입니다. 대화를 주고받을 수 없어요.

### 7.2 Fiat-Shamir 변환

**Fiat-Shamir heuristic** (1986)이 이 문제를 해결합니다:

> "검증자의 랜덤 챌린지를 **해시 함수**로 대체한다"

증명자가:

1. 첫 메시지를 만듦

2. 그 메시지의 해시를 "랜덤 챌린지"로 사용
3. 응답을 만듦
4. 다음 해시를 다음 챌린지로 사용
5. ...

해시 함수가 랜덤 오라클처럼 행동한다면, 증명자가 챌린지를 예측하거나 조작할 수 없습니다.

결과: **Non-Interactive Zero-Knowledge Proof (NIZK)**

블록체인에서 쓰는 ZKP는 전부 이 방식입니다. 증명자가 증명을 생성해서 제출하면, 누구나 (스마트 컨트랙트 포함) 독립적으로 검증할 수 있습니다.

---

## 8. SNARK: 간결한 증명

**SNARK**는 "Succinct Non-interactive ARgument of Knowledge"의 약자입니다.

- **Succinct (간결함)**: 증명 크기가 작고, 검증이 빠르다
- **Non-interactive**: 한 번에 증명을 제출하면 끝
- **ARgument**: 계산적 건전성 (무한한 계산 능력이 있으면 깰 수 있음)
- **of Knowledge**: 증명자가 실제로 witness를 "알고 있음"을 보장

"Succinct"가 핵심입니다. 증명 크기가 원래 계산의 크기와 **무관하게** 작습니다.

| 시스템 | 증명 크기 | 검증 시간 |
|---|---|---|
| Groth16 | 188 bytes | ~수 밀리초 |
| PLONK | ~400 bytes | ~수 밀리초 |
| STARK | ~수십 KB | ~수십 밀리초 |

수백만 개의 제약 조건을 가진 회로도, 증명은 수백 바이트입니다.

---

## 9. 마치며: 앞으로의 여정

이번 편에서 다룬 것:

- ZKP의 핵심 개념 (레드스톤 비유)
- 세 가지 조건: 완전성, 건전성, 영지식성
- 왜 블록체인에서 중요한가
- 산술 회로의 기본 아이디어
- Interactive에서 Non-interactive로

다음 편에서는 **유한체와 타원곡선**을 다룹니다. ZKP가 작동하려면 "컴퓨터가 다룰 수 있는 수의 세계"가 필요합니다:

- 왜 무한이 아니라 유한인가
- 왜 소수여야 하는가
- 타원곡선이 왜 등장하는가

이 수학적 기반을 이해해야 3편의 산술화(Arithmetization)가 어떻게 가능한지 알 수 있습니다.

---

## References


1. Goldwasser, S., Micali, S., & Rackoff, C. (1985). *The Knowledge Complexity of Interactive Proof Systems*. STOC '85.
2. Fiat, A., & Shamir, A. (1986). *How to Prove Yourself: Practical Solutions to Identification and Signature Problems*. CRYPTO '86.
3. Buterin, V. (2016). *Quadratic Arithmetic Programs: from Zero to Hero*. Medium.
4. Thaler, J. (2022). *Proofs, Arguments, and Zero-Knowledge*. Online textbook.